

Se você consegue gerar uma demonstração narrativa desse projeto explicando cada etapa do que foi projetado e como foi desenvolvido

Com certeza! Preparei uma demonstração narrativa do projeto, explicando cada etapa e como ela foi desenvolvida. É um resumo detalhado que você pode usar como base para sua apresentação ou para entender o fluxo do trabalho:

Demonstração Narrativa do Projeto: DocuDigit - Classificador de Dígitos Manuscritos

Olá a todos! Hoje vou apresentar nosso projeto, o **DocuDigit**, um pipeline de Machine Learning multiclasse desenvolvido para a classificação de dígitos manuscritos. Nosso objetivo foi construir um sistema robusto que não só classificasse os dígitos com alta precisão, mas também demonstrasse sua capacidade de generalização e como ele se comporta diante de dados inesperados.

Fase 1: Carregamento e Análise Exploratória de Imagens (EDA)

Começamos o projeto com a **Fase 1**, focada no carregamento e na Análise Exploratória de Dados (EDA) do dataset MNIST. Utilizamos a função `fetch_openml` do `scikit-learn` para carregar o conjunto de dados `mnist_784`, que nos forneceu `X` com 70.000 imagens e 784 características (pixels) por imagem, e `y` com os 70.000 rótulos correspondentes. Essa etapa inicial é crucial para entender a matéria-prima do nosso sistema.

Verificamos a **distribuição das classes** e confirmamos que o dataset é razoavelmente balanceado, com todos os dígitos (0 a 9) tendo um número similar de ocorrências. Isso é importante para evitar que o modelo seja viciado em uma ou outra classe. Em seguida, para ter uma noção visual dos dados, plotamos uma **grade de exemplos de imagens** para cada dígito. Isso nos mostrou a variabilidade da escrita manual e a representação de cada dígito em uma matriz 28x28 pixels, com valores de intensidade de 0 a 255.

Finalizamos a EDA com uma **interpretação da estrutura dos dados**, explicando que cada imagem 28x28 é "achatada" em um vetor de 784 características, que é o formato esperado pela maioria dos algoritmos de Machine Learning.

Fase 2: Pipeline de Pré-processamento e Divisão dos Dados

A **Fase 2** foi dedicada a preparar os dados para o treinamento. Primeiro, realizamos a **divisão estratificada dos dados** em conjuntos de treino (70%), validação (10%) e teste (20%). A estratificação (`stratify=y`) foi fundamental para garantir que a proporção de cada dígito fosse mantida em todos os conjuntos, preservando o balanceamento original do dataset. Obtivemos `X_train` com 49.000 amostras, `X_val` com 7.000 e `X_test` com 14.000.

Em seguida, aplicamos a **normalização dos pixels**, escalando os valores de 0-255 para o intervalo de 0.0-1.0. Justificamos essa escolha explicando que a normalização é vital para a **convergência mais rápida** de algoritmos baseados em gradientes (como Redes Neurais),

melhora o desempenho de **modelos baseados em distância** (como SVM) e **reduz o viés**, garantindo que nenhum pixel domine a aprendizagem apenas por ter uma escala maior.

Fase 3: Implementação e Treinamento dos 3 Modelos

Na **Fase 3**, selecionamos e treinamos três modelos de classificação distintos para o nosso problema:

1. **Support Vector Machine (SVM):** Escolhemos o SVM por sua eficácia em espaços de alta dimensão e sua capacidade de encontrar um hiperplano ótimo. Configuramos com $C=100$ para uma menor regularização (buscando alta acurácia no treino) e `kernel='rbf'` para capturar relações não lineares. O treinamento levou cerca de **206 segundos**, alcançando uma acurácia de validação de **0.9814**.
2. **Random Forest Classifier:** Este é um algoritmo de ensemble robusto, conhecido por controlar o overfitting. Definimos `n_estimators=200` para mais árvores (melhor generalização) e `max_depth=20` para limitar a complexidade de cada árvore. Foi o mais rápido para treinar, com **65.91 segundos**, mas obteve uma acurácia de validação de **0.9667**.
3. **Multi-layer Perceptron (MLP) Classifier:** Uma rede neural feedforward, simples mas poderosa para aprender relações não lineares. Usamos uma camada oculta com 100 neurônios (`hidden_layer_sizes=(100,)`), função de ativação `relu` e o otimizador `adam`. O treinamento durou **100.78 segundos**, com uma acurácia de validação de **0.9749**.

Fase 4: Avaliação Comparativa de Desempenho

A **Fase 4** foi dedicada à avaliação final dos modelos no conjunto de teste, que eles nunca haviam visto. Geramos **matrizes de confusão** e **relatórios de classificação** para cada um. As matrizes nos revelaram padrões de confusão comuns, como dígitos visualmente semelhantes (e.g., 4 e 9, 2 e 7, 3 e 5).

Consolidamos todas as métricas (Acurácia Global, Precisão, Revocação, F1-Score e Tempo de Treinamento) em uma **tabela comparativa**. Concluímos que o **SVM** foi o mais preciso (0.9830 de acurácia), mas também o mais lento. O **MLP** ofereceu um excelente balanço entre tempo de treinamento e acurácia (0.9769), tornando-se uma alternativa muito forte. O Random Forest, embora rápido, ficou ligeiramente atrás em performance.

Fase 5: Testes de Generalização Extrema (Out-of-Distribution - OOD) e Inferência com Imagens Manuscritas Próprias

Esta foi a fase mais desafiadora e reveladora:

- **Desafio (A) Treinamento Restrito com Classes Ocultadas (Class Masking):** Retreinamos um modelo MLP (chamado `mlp_model_masked`) sem expô-lo aos dígitos '4' e '7' no treinamento. O conjunto de treino foi reduzido para 39.118 amostras, e a acurácia de validação caiu para 0.7819, como esperado, pois o modelo não tinha como classificar corretamente as classes ocultas no conjunto de validação.
- **Desafio (B) Teste de Generalização Extrema (Inferência OOD):** Em seguida, testamos o `mlp_model_masked` *exclusivamente* com imagens dos dígitos '4' e '7' do

conjunto de teste ($X_{\text{test_ood}}$). A **matriz de confusão OOD** foi esclarecedora: o modelo tentou classificar os '4's e '7's como outras classes que ele conhecia (como '9', '2', '3'), demonstrando o conceito de "**falsa certeza**". Ele não tem a capacidade de dizer "não sei" e faz previsões errôneas, mas confiantes, sobre dados fora de sua distribuição de treinamento. Isso sublinha a importância da detecção de OOD em sistemas de IA.

- **Desafio (C) Inferência com Imagens Manuscritas Próprias:** Para este desafio prático, implementamos um **pipeline robusto de pré-processamento** para nossas próprias imagens manuscritas (CHARACTER0.png a CHARACTER9.png). Este pipeline realizou conversão para escala de cinza, inversão de cores (para fundo preto, dígito branco), redimensionamento e centralização do dígito em um canvas 28x28, e finalmente a normalização. Em seguida, usamos nosso `mlp_model` (treinado com todas as classes) para prever os dígitos. A demonstração visual mostrou as previsões do modelo e suas probabilidades, evidenciando o quão bem ele generaliza para novas entradas, embora algumas previsões possam ter probabilidades mais baixas devido à variação na escrita manual.

Em suma, o projeto **DocuDigit** nos permitiu não apenas construir e comparar modelos de classificação de dígitos com alto desempenho, mas também explorar conceitos avançados como generalização extrema e a crucial necessidade de pré-processamento para dados do mundo real. Este pipeline serve como uma base sólida para futuras melhorias e aplicações em sistemas que demandam a interpretação de dados visuais manuscritos.

Espero que esta narrativa seja útil para você! Não hesite em me perguntar se precisar de mais detalhes ou ajustes.